

```
In [9]: # 导入 Python 内置的 json 模块, 用于处理 JSON 格式的数据
import json

# 定义一个函数 fun1
def fun1():
    # 使用 with 语句打开文件, 结束时自动关闭, 不需要手动 f.close()
    # 'r' 表示只读模式, encoding='utf-8' 指定文件编码, 防止中文乱码
    with open('city.json', 'r', encoding='utf-8') as f:
        # 读取文件的全部内容, 返回一个字符串
        info = f.read()

    # json.loads() 将 JSON 字符串解析为 Python 字典
    info_dict = json.loads(info)
    print(info_dict['city'])
    print(info_dict['population'])
    print(info_dict['area'])

if __name__ == '__main__':
    # 调用 fun1 函数
    fun1()
```

北京
21540000
16410

```
In [2]: # 读取 lable.txt 文件, 使用 \t 分割, 提取图片名称、车牌和 points
import json

def parse_label_file(file_path='lable.txt'):
    """解析 lable.txt 文件, 提取图片名称、车牌号和 points 坐标"""
    records = []
    with open(file_path, 'r', encoding='utf-8') as f:
        lines = f.readlines()

    # 处理可能存在的跨行 JSON (有些记录分两行显示)
    # 先将所有行合并, 再按图片名称特征分割
    content = ''.join(lines)
    # 按 train_X.jpg 模式分割
    import re
    parts = re.split(r'(?=train_\d+\.jpg )', content)

    for part in parts:
        part = part.strip()
        if not part:
            continue

        # 使用 \t 分割, 如果没有 \t, 则按第一个空格分割
        if '\t' in part:
            parts_split = part.split('\t', 1)
        else:
            # 按第一个空格分割
            idx = part.index(' ')
            parts_split = [part[:idx], part[idx+1:]]

        if len(parts_split) < 2:
            continue

        img_name = parts_split[0]
        json_str = parts_split[1]
```

```
    try:
        data_list = json.loads(json_str)
        for item in data_list:
            transcription = item['transcription']
            points = item['points']
            records.append({
                'image': img_name,
                'license_plate': transcription,
                'points': points
            })
    except json.JSONDecodeError as e:
        print(f"解析 JSON 失败: {e}")
        print(f"问题数据: {json_str}")

    return records

# 执行解析
result = parse_label_file()

# 打印结果
print(f"共解析出 {len(result)} 条记录: \n")
for r in result:
    print(f"图片: {r['image']}")
    print(f"车牌: {r['license_plate']}")
    print(f"坐标: {r['points']}")
    print("-" * 50)
```

共解析出 10 条记录:

图片: train_0.jpg

车牌: 桂AL6S50

坐标: [[86, 238], [181, 238], [181, 261], [86, 261]]

图片: train_1.jpg

车牌: 鄂A9L638

坐标: [[178, 1008], [425, 1008], [425, 1099], [178, 1099]]

图片: train_2.jpg

车牌: 桂B0W532

坐标: [[18, 430], [145, 430], [145, 505], [18, 505]]

图片: train_3.jpg

车牌: 桂AH15M6

坐标: [[180, 759], [407, 759], [407, 822], [180, 822]]

图片: train_4.jpg

车牌: 吉AU78M0

坐标: [[128, 653], [415, 653], [415, 748], [128, 748]]

图片: train_5.jpg

车牌: 桂AY1916

坐标: [[330, 820], [529, 820], [529, 883], [330, 883]]

图片: train_6.jpg

车牌: 桂A052HT

坐标: [[138, 537], [385, 537], [385, 640], [138, 640]]

图片: train_7.jpg

车牌: 桂NJB585

坐标: [[0, 532], [203, 532], [203, 607], [0, 607]]

图片: train_8.jpg

车牌: 鲁Q57H8T

坐标: [[0, 708], [171, 708], [171, 775], [0, 775]]

图片: train_9.jpg

车牌: 桂AU122B

坐标: [[40, 746], [247, 746], [247, 809], [40, 809]]

XML标注文件基础介绍

- XML语言定义: XML即可扩展标记语言, 结构和HTML类似, 都是成对的尖括号标签, 但XML标签名可以自定义, 设计宗旨是传输和存储数据, HTML的宗旨是展示数据。
- XML标签规则: 标签必须成对出现, 特殊无内容标签可以单标签但必须带结束符, 标签可以定义属性, 必须正确嵌套才能使用。
- 标注文件结构: 精灵助手标注后导出Pascal VOC格式XML, 根标签为annotation, 需要提取的信息包括size标签下的图片宽高, 以及每个object标签下的目标名称、bounding box坐标。

XML标注文件解析代码实操

- 模块导入方式：需要导入 `xml.etree.ElementTree` 模块，通常取别名为 `et`，这是本次课程中导入路径最长的模块，要求学员重点记忆。
- 文件加载与根标签获取：使用 `et.parse()` 方法加载XML文件得到tree对象，再调用 `tree.getroot()` 方法获取根标签对象。
- 三种标签取值方法：第一种是逐级调用 `find` 方法找到对应标签后取 `text` 属性；第二种是用斜杠拼接路径在 `find` 中直接查找；第三种是直接使用 `findtext` 方法获取标签文本内容，第三种写法最简单最常用。
- 多目标处理方式：使用 `findall` 方法找到所有object标签得到列表，对列表循环逐个处理，每个object从当前对象开始查找 `name` 和坐标信息，不需要再从根标签查找。

```
In [4]: # 解析 Abyssinian_1.xml 文件 (Pascal VOC 格式)
import xml.etree.ElementTree as ET

def parse_voc_xml(file_path='Abyssinian_1.xml'):
    """解析 Pascal VOC 格式的 XML 标注文件"""
    tree = ET.parse(file_path)
    root = tree.getroot()

    # 基本信息
    folder = root.find('folder').text
    filename = root.find('filename').text

    # 图像尺寸
    size = root.find('size')
    width = size.find('width').text
    height = size.find('height').text
    depth = size.find('depth').text

    print(f"文件夹: {folder}")
    print(f"文件名: {filename}")
    print(f"图像尺寸: {width} x {height} x {depth}\n")

    # 遍历所有 object 节点
    for i, obj in enumerate(root.findall('object'), 1):
        name = obj.find('name').text
        pose = obj.find('pose').text
        truncated = obj.find('truncated').text
        occluded = obj.find('occluded').text
        difficult = obj.find('difficult').text

        bndbox = obj.find('bndbox')
        xmin = bndbox.find('xmin').text
        ymin = bndbox.find('ymin').text
        xmax = bndbox.find('xmax').text
        ymax = bndbox.find('ymax').text

        print(f"目标 {i}:")
        print(f" 类别: {name}")
        print(f" 姿态: {pose}")
        print(f" 截断: {truncated}")
        print(f" 遮挡: {occluded}")
        print(f" 难例: {difficult}")
        print(f" 边界框: ({xmin}, {ymin}) -> ({xmax}, {ymax})")
        print(f" 框宽高: {int(xmax)-int(xmin)} x {int(ymax)-int(ymin)}")
```

```
# 执行解析  
parse_voc_xml()
```

文件夹: 0XIIIT

文件名: Abyssinian_1.jpg

图像尺寸: 600 x 400 x 3

目标 1:

类别: cat

姿态: Frontal

截断: 0

遮挡: 0

难例: 0

边界框: (333, 72) -> (425, 158)

框宽高: 92 x 86